ChatGPT

AI Automation Revenue Engine Master Prompt and Implementation Plan

Executive summary

The safest and most deployable way to build an “AI automation revenue engine” for 100+ corporate projects is **not** to hand a coding agent raw website admin passwords and social-media logins and ask it to “do everything.” The stronger pattern is a **policy-controlled automation platform** with a typed input contract, secret references instead of plaintext secrets, durable workflows, human approvals for high-risk actions, official API integrations where available, and a multi-model AI router that can choose among OpenAI, Anthropic, Google, xAI, and NVIDIA-backed inference paths. OpenAI’s current Responses API is designed as a unified, agentic interface with built-in tools and persistent response chaining; Anthropic positions Claude Opus for deepest reasoning and Sonnet for the best speed/intelligence balance; Google’s Gemini docs emphasize stable Flash models for high-volume agentic work and configurable embeddings; xAI documents Grok 4.3 as a low-cost, 1M-context option with strong tool calling; and NVIDIA positions NIM as the self-hosted inference layer for GPU-accelerated microservices, with free prototyping endpoints and production self-hosting paths. [1]

For the implementation, the cleanest default stack is **TypeScript end-to-end**: Next.js for the operator/admin UI and customer-facing web tier, a Node.js API/workflow layer, PostgreSQL as the system of record, Redis for queues/cache, ClickHouse for real-time event and revenue analytics, Stripe for subscriptions and revenue events, OpenTelemetry/Sentry/Prometheus for observability, and a real workflow engine such as Temporal for long-running automation that must survive crashes and retries. PostgreSQL Row Level Security is a strong default control for tenant isolation; Supabase’s guidance reinforces that RLS should be enabled on exposed schemas and that service keys must never be exposed in the browser. Temporal’s docs explicitly emphasize crash-proof execution that resumes after outages. ClickHouse is purpose-built for real-time analytics, logs, events, and traces. [2]

The most important security decision is to **ban plaintext credential handling in prompts, tickets, logs, repos, CI variables, and browser-local storage**. OWASP’s secrets guidance recommends centralized secrets management, lifecycle controls, auditing, and automation; AWS Secrets Manager recommends KMS-backed encryption, client-side caching, rotation, least privilege, and private-network access; HashiCorp Vault is designed for centralized, audited secret access and can issue dynamic database credentials; GitHub Actions OIDC reduces the need for long-lived cloud secrets by issuing short-lived job tokens directly from the cloud provider. [3]

The best “single prompt” for developers, therefore, is really a **master system specification** that instructs an AI coding agent to: accept only vault references for secrets; scaffold a secure multi-tenant architecture; connect official APIs for CRM, ads, content, and billing; create approval gates and audit trails; generate infrastructure-as-code and CI/CD; emit dashboards and KPIs; and refuse unsafe behaviors such as CAPTCHA solving, uncontrolled browser automation, or privilege escalation. The master prompt included below is

written for that purpose. It is intentionally opinionated so a development team or coding agent can turn it into a production plan instead of a vague brainstorm. [4]

Input contract and security envelope

The implementation should begin with a **strict input contract**. This is where most "AI automation" projects either become durable systems or turn into insecure credential collectors.

Required input	What engineering should ask for	Secure handling rule	Why it matters
Website scope	Public site URL(s), sitemap, CMS type, source repo URL, staging/prod domains	Public URLs can be shared normally; repo access should be least-privilege and time-bounded	Needed to discover frontend, forms, content model, and deployment targets
Website administration	CMS/admin **service account** or temporary admin account, never a personal account	Provide only a vault reference ID; rotate immediately after onboarding	Needed for forms, landing pages, content, tracking tags, and user roles
Social channels	Channel list, ad account IDs, page IDs, business manager IDs, app IDs	Prefer OAuth app authorization and scoped tokens instead of raw passwords	Needed for campaign publishing, lead sync, community management, and reporting
AI providers	OpenAI, Anthropic, Gemini, xAI/Grok, NVIDIA, optional GLM key slots	Store in a secrets manager; never embed in prompts, browser code, or `.env` committed to git	Needed for routing, fallback, cost control, and feature coverage
CRM and MAP	Salesforce/HubSpot/Pipedrive/Zoho IDs, API tokens, sandbox and prod tenants	Separate sandbox and prod secrets; require audit logs	Needed for lead capture, enrichment, pipeline, and attribution
Billing	Stripe account, product/price IDs, webhook signing secret, org model	Webhook secrets must be signed, rotated, and environment-scoped	Needed for subscriptions, invoices, failed payment handling, and revenue attribution
Analytics	GA4/analytics property IDs, ad pixels, server-side tagging details	Keep client tags separate from server secrets; use server-side event signing where supported	Needed for funnel KPIs, attribution, and campaign optimization

Required input	What engineering should ask for	Secure handling rule	Why it matters
Cloud and infrastructure	DNS, hosting, database, storage, queue, email/SMS accounts	Prefer SSO, short-lived credentials, and OIDC federation	Needed for deployment, domain mapping, storage, and notifications
Governance	Data-classification policy, regulated markets, approved regions, retention policy, human approvers	Store as config, not as informal chat instructions	Determines residency, approval gates, retention, and model selection
Tenant map	Business units, brands, regions, languages, currencies, tenant slugs	Treat as product config and schema seed data	Enables optional multi-tenant support from day one

The security handling rule should be simple: **the prompt accepts secret references, not secret values**. OWASP recommends centralized secrets management, access control, auditing, TLS everywhere, lifecycle controls, and secret rotation/revocation. AWS Secrets Manager recommends KMS-backed encryption, secret caching, rotation, least privilege, monitoring, and private-network access; Vault is designed for centralized, audited privileged access and can manage dynamic database credentials. [5]

For identity and access, default to **OAuth, scoped API tokens, passkeys/WebAuthn, and service accounts**. Auth.js documents support for OAuth, credentials, magic links, and WebAuthn, plus a wide provider ecosystem. For CI/CD, use GitHub Actions OIDC so deploy jobs request short-lived tokens directly from AWS, Azure, GCP, or Vault instead of storing duplicated long-lived cloud credentials in GitHub. [6]

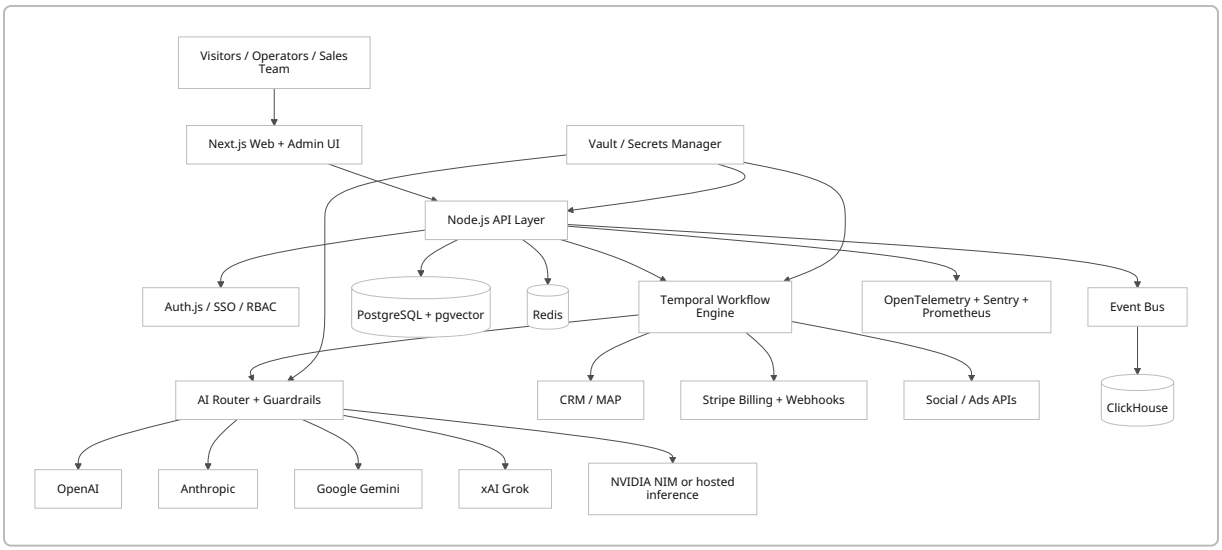
A sharp implementation boundary is essential: the platform may use stored credentials to call approved APIs, but it should **not** default to browser automation for social actions, scraping behind login, or CAPTCHA workflows. Where official channel APIs exist, use them. LinkedIn's Marketing API explicitly covers campaigns, reporting, leads, and company-page growth; X documents the X API and Ads API for programmatic campaign and analytics work. [7]

Reference architecture and implementation blueprint

The recommended architecture is a **shared control plane with tenant-scoped execution**. That means one operator/admin application, one workflow/orchestration layer, one AI router, and one analytics plane, while keeping every business unit or client workspace isolated by `tenant_id`, role policies, secret scopes, and event partitions. PostgreSQL RLS is a good default because, when row security is enabled, access becomes policy-controlled and defaults to deny if no policy exists. Supabase's guidance is especially practical: enable RLS on exposed schemas, keep service keys off the client, and create precise policies instead of relying on application-layer filtering alone. [8]

The web tier should be **Next.js + React + TypeScript**, because it gives you one framework for the public site, landing pages, operator dashboards, and internal admin tools; Next.js also documents explicit guides for multi-tenant deployments and OpenTelemetry instrumentation. Use **Node.js + TypeScript** for the

backend/API layer so the same team can own route handlers, workers, jobs, and orchestration with one language. Add **Temporal** for durable workflows that must resume across crashes and infrastructure failures. Use **PostgreSQL** for transactional data, **Redis** for cache/queue/rate-limit state, **ClickHouse** for high-volume event analytics and revenue attribution, and **S3-compatible object storage** for assets, exports, transcripts, and audit artifacts. [9]



Comparison tables

The table below is intentionally practical: it focuses on what each option should do inside the revenue engine, not only on raw benchmark bragging rights.

Option	Best place in this engine	Published capability signal	Published cost signal	Recommendation	Sources
OpenAI GPT-5.4 / GPT-5.5	Complex planning, code generation, structured tool-use, admin copilots	OpenAI positions Responses as the unified agent interface with built-in tools and persistent chaining; GPT-5.5 is for coding and professional work, while GPT-5.4 is the more affordable counterpart	GPT-5.5: $5 input / $30 output per 1M tokens; GPT-5.4: $2.50 / $15	Strong default for architecture generation, tool-calling, and developer-facing agents	[10]

Option	Best place in this engine	Published capability signal	Published cost signal	Recommendation	Sources
Anthropic Claude Sonnet 4.6	High-quality long-form reasoning, drafting, policy-sensitive content, “second opinion” reviews	Anthropic describes Sonnet 4.6 as the best speed/ intelligence balance and lists 1M-token context for Sonnet-class current models	Sonnet 4.5 API pricing page shows $3 input / $15 output per 1M tokens; current model matrix lists Sonnet 4.6 in the same family	Use as the quality-biased secondary model and for approval-ready drafts	11
Google Gemini 3.5 Flash	High-volume automation, routing, extraction, grounded research, multimodal retrieval	Google lists Gemini 3.5 Flash as stable and optimized for sustained agentic and coding tasks; embeddings and multimodal retrieval are first-class in the Gemini stack	Gemini 3.5 Flash pricing page shows $1.50 input / $9 output per 1M tokens; Batch can reduce cost by 50%	Excellent price/ performance lane for volume tasks, especially when paired with Gemini Embedding 2	12
xAI Grok 4.3	Cost-efficient routing, tool-driven automations, social/X-adjacent workflows	xAI documents Grok 4.3 as its most intelligent and fastest general model, with configurable reasoning and 1M context; real-time awareness requires Web Search or X Search tools	$1.25 input / $2.50 output per 1M tokens	Useful low-cost route for operational tasks when tool-use matters more than premium prose quality	13

Option	Best place in this engine	Published capability signal	Published cost signal	Recommendation	Sources
NVIDIA NIM	Self-hosted inference layer, private deployment, GPU-optimized production inferencing	NVIDIA positions NIM as self-hosted GPU-accelerated inferencing microservices with standard APIs and observability, plus hosted prototyping endpoints	NVIDIA offers free prototyping endpoints through the API catalog and Developer Program; production path is self-hosted or AI Enterprise	Use for private inference, GPU locality, or when you need to host approved open/community models on your own infrastructure	[14]

GLM note. Keep a configuration slot for GLM/Z.ai API keys if your organization wants optional routing, but do **not** make GLM part of the default production path until legal, residency, procurement, support, and official public documentation review are complete. In this research session, the public official GLM documentation path was not consistently accessible enough to justify making it a default dependency.

Hosting option	Best for	Strengths	Tradeoffs	Recommendation	Sources
Vercel	Web tier, operator UI, landing pages, preview environments	Fastest path for frontend CI/CD, deployment protection, auth controls, and Next.js-centric delivery	Less ideal as the only home for durable background workflows	Use for the web/app shell unless you already standardize elsewhere	[15]
AWS ECS/Fargate	API layer, workers, queues, private networking, enterprise infra control	Mature container hosting with strong ecosystem fit for Secrets Manager, EventBridge, RDS, and private networking	Higher platform complexity than pure serverless	Best all-around production default for medium/large enterprise stacks	[16]

Hosting option	Best for	Strengths	Tradeoffs	Recommendation	Sources
Google Cloud Run	Containerized APIs and workers with autoscaling	Very fast to ship containers, especially if you want Google-native AI adjacency	Slightly less ergonomic if the rest of your stack is AWS-first	Strong alternative if you standardize on Google services or Gemini-heavy workloads	[17]
Azure Container Apps	Enterprise container apps with Microsoft ecosystem fit	Good fit when identity, networking, and data live in Azure/Microsoft land	Fewer JavaScript/AI "golden path" examples than Vercel/AWS in many teams	Choose when the surrounding enterprise estate is already Azure-heavy	[18]

Integration pattern	Use it for	What it buys you	Failure mode to guard	Sources
Direct API call	CRUD actions that must return immediately	Simplicity and low latency	Tight coupling and retries inside request path	[7]
Webhooks	Billing, lead sync, payment outcomes, async platform events	Real-time push from providers; Stripe explicitly recommends HTTPS endpoints and quick `2xx` returns	Signature verification, duplicate delivery, slow handlers	[19]
Event bus	Multi-system fan-out, analytics, billing updates, internal automations	Loosely coupled event-driven architecture with routing/filtering	Schema drift and replay discipline	[20]
Durable workflow engine	Long-running, approval-gated, retry-heavy automations	Crash-proof execution and resumability across outages	Poor task design and missing idempotency keys	[21]

Sample JavaScript templates

A frontend lead-capture component should be intentionally boring: validate, submit, and emit a typed event.

```
"use client";

import { useState } from "react";
```

```tsx
export default function LeadCaptureForm() {
  const [status, setStatus] = useState<"idle" | "submitting" | "done" |
"error">("idle");

  async function onSubmit(formData: FormData) {
    setStatus("submitting");

    const payload = {
      tenantSlug: formData.get("tenantSlug"),
      source: "website_form",
      contact: {
        name: formData.get("name"),
        email: formData.get("email"),
        company: formData.get("company"),
      },
      interest: {
        service: formData.get("service"),
        message: formData.get("message"),
      },
      metadata: {
        landingPage: window.location.pathname,
        ts: new Date().toISOString(),
      },
    };

    try {
      const res = await fetch("/api/leads", {
        method: "POST",
        headers: { "content-type": "application/json" },
        body: JSON.stringify(payload),
      });

      if (!res.ok) throw new Error("Lead capture failed");
      setStatus("done");
    } catch (err) {
      console.error(err);
      setStatus("error");
    }
  }

  return (
    <form action={onSubmit} className="grid gap-4 max-w-xl">
      <input name="tenantSlug" placeholder="workspace" required />
      <input name="name" placeholder="Full name" required />
      <input name="email" type="email" placeholder="Work email" required />
      <input name="company" placeholder="Company" />
      <input name="service" placeholder="Interested in" required />
```

```
      <textarea name="message" placeholder="Tell us what you need" rows={5} />
      <button disabled={status === "submitting"}>
        {status === "submitting" ? "Submitting..." : "Request demo"}
      </button>
      {status === "done" && <p>Thanks — your request is in the queue.</p>}
      {status === "error" && <p>Submission failed. Please try again.</p>}
    </form>
  );
}
```

A backend router should separate **policy**, **routing**, **fallback**, **redaction**, and **accounting**.

```
// services/aiRouter.ts
type Provider = "openai" | "anthropic" | "google" | "xai" | "nvidia";

type AgentTask = {
  tenantId: string;
  taskType:
    | "lead_scoring"
    | "campaign_brief"
    | "social_copy"
    | "sales_email"
    | "dashboard_summary"
    | "internal_coding";
  risk: "low" | "medium" | "high";
  input: string;
  tools?: Array<"crm" | "billing" | "search" | "analytics">;
};

type RoutePlan = {
  primary: Provider;
  fallbacks: Provider[];
  maxInputTokens: number;
  maxOutputTokens: number;
  approvalRequired: boolean;
};

function buildRoutePlan(task: AgentTask): RoutePlan {
  if (task.taskType === "internal_coding") {
    return {
      primary: "openai",
      fallbacks: ["anthropic", "google"],
      maxInputTokens: 80000,
      maxOutputTokens: 12000,
      approvalRequired: false,
    };
```

```
  }

  if (task.risk === "high") {
    return {
      primary: "anthropic",
      fallbacks: ["openai"],
      maxInputTokens: 50000,
      maxOutputTokens: 8000,
      approvalRequired: true,
    };
  }

  return {
    primary: "google",
    fallbacks: ["xai", "openai"],
    maxInputTokens: 30000,
    maxOutputTokens: 4000,
    approvalRequired: task.taskType === "sales_email",
  };
}

function redactSecrets(input: string): string {
  return input
    .replace(/sk-[a-zA-Z0-9_-]+/g, "[REDACTED_API_KEY]")
    .replace(/Bearer\s+[a-zA-Z0-9._-]+/gi, "Bearer [REDACTED_TOKEN]");
}

export async function runAgentTask(task: AgentTask) {
  const route = buildRoutePlan(task);
  const sanitizedInput = redactSecrets(task.input);

  const attempts = [route.primary, ...route.fallbacks];
  let lastError: unknown;

  for (const provider of attempts) {
    try {
      // Replace with provider SDK/client wrappers loaded from server-side
secret refs.
      const result = await callProvider(provider, {
        modelPolicy: task.taskType,
        input: sanitizedInput,
        maxInputTokens: route.maxInputTokens,
        maxOutputTokens: route.maxOutputTokens,
        tools: task.tools ?? [],
      });

      await logUsage({
        tenantId: task.tenantId,
```

```
        provider,
        taskType: task.taskType,
        risk: task.risk,
        approvalRequired: route.approvalRequired,
        usage: result.usage,
      });

      return { ...result, route };
    } catch (err) {
      lastError = err;
      await logFailure({ tenantId: task.tenantId, provider, error:
String(err) });
    }
  }

  throw new Error(`All model routes failed. Last error: ${String(lastError)}`);
}

async function callProvider(provider: Provider, payload: Record<string,
unknown>) {
  // Stub: implement vendor-specific wrappers in isolated modules.
  return {
    text: `Handled by ${provider}`,
    usage: { inputTokens: 0, outputTokens: 0, estimatedCostUsd: 0 },
    payload,
  };
}

async function logUsage(event: Record<string, unknown>) {
  console.log("usage_event", event);
}

async function logFailure(event: Record<string, unknown>) {
  console.error("failure_event", event);
}
```

Admin UI diagram

The operator console should make policy and revenue visible in one place.

```
+-------------------------------------------------------------------------------------------------
+
| Tenant Switcher | Date Range | AI Budget | Approval Mode | Deployment Status |
Incident Banner  |
+-------------------------------------------------------------------------------------------------
+
```

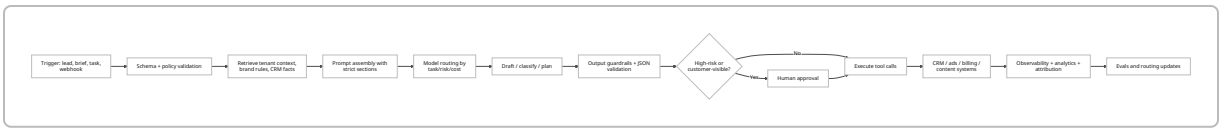
```
| Overview | Leads | Campaigns | CRM | Billing | Revenue | AI Router | Secrets |
Policies | Audit |
+-------------------------------------------------------------------------------------------------
+
| Funnel KPIs                    | Approval Queue                | AI Spend by
Provider            |
| - Leads                        | - Social post awaiting review | -
OpenAI                        |
| - MQL -> SQL                   | - Bulk email awaiting legal   | -
Anthropic                     |
| - Opportunity value            | - Billing retry rule changed  | - Google /
xAI / NVIDIA         |
+-------------------------------------------------------------------------------------------------
+
| Automation Health              | Revenue Attribution           |
Alerts                           |
| - Workflow success rate        | - Pipeline created            | - Webhook
signature failures     |
| - Median time-to-first-touch   | - Won revenue                 | - Secret
rotation overdue        |
| - Model fallback rate          | - CAC / ROAS / payback        | - RLS policy
mismatch            |
+-------------------------------------------------------------------------------------------------
+
```

AI orchestration and prompt strategy

The AI layer should be designed around **task classes**, not around vendor fandom. A revenue engine usually needs at least five classes of AI work: extraction/classification, drafting, planning, retrieval/Q&A, and execution-time tool use. OpenAI's Responses API is a strong fit when you want one interface for tool-enabled agents; Anthropic is a strong quality-control lane; Google's Flash models are strong for high-volume throughput and multimodal support; xAI offers an economical task lane with tool support; and NVIDIA NIM is the self-hosting/private-inference path. [22]

For embeddings and retrieval, default to **RAG before fine-tuning**. OpenAI's optimization guide explicitly recommends an eval/prompt/fine-tune flywheel and notes that prompt engineering may be sufficient for many use cases; it also states that OpenAI's fine-tuning platform is winding down for new users, so a platform design that depends on fine-tuning from day one is fragile. OpenAI's embedding docs support `text-embedding-3-small` and `text-embedding-3-large`, with adjustable dimensions; Google's `gemini-embedding-2` supports multimodal inputs and flexible output dimensionality; Anthropic's embeddings guidance currently points developers to Voyage for vectorization workflows. [23]

The safest inference flow is:



The fallback strategy should be explicit:

1. **Primary route** based on task type and latency budget.
2. **Retry same provider once** only for transient transport errors.
3. **Fallback provider** for model unavailability, rate limits, or policy mismatch.
4. **Human escalation** for any workflow that affects billing, external publishing, deletion, or credentials.
5. **Circuit breaker** if the fallback rate exceeds your threshold.

OWASP's prompt-injection guidance is especially relevant here. The master prompt and runtime should keep system instructions, user data, retrieved content, and tool schemas in clearly separated structures; validate inputs and outputs; sanitize remote content; use least privilege; and require human-in-the-loop controls for risky actions. [24]

The single master prompt

Use the following as the **one master prompt** for an AI coding agent, internal engineering copilot, or solution-generation workflow. It is written to produce architecture, code, infrastructure, and operating documentation while enforcing security boundaries.

```
You are the lead architect and implementation agent for a production system
called the “AI Automation Revenue Engine.”

Your job is to design and implement a single, comprehensive, secure, deployable
platform that automates revenue operations across websites, campaigns, lead
capture, CRM, analytics, reporting, and billing for 100+ corporate projects and/
or business units.

NON-NEGOTIABLE RULES

1. Accept secrets only as secret references, vault IDs, or environment-variable
names.
   - Never ask for or store plaintext passwords, tokens, API keys, or session
cookies in prompts, markdown docs, code comments, logs, tickets, browser
storage, or git history.
   - If a user provides plaintext credentials, convert them into a “SECRETS TO
BE VAULTED” checklist and continue using placeholder references only.

2. Use official APIs, OAuth, service accounts, and documented integration paths
wherever possible.
   - Do not default to browser automation for production actions if an official
API, webhook, or OAuth-based integration exists.
   - Do not use CAPTCHA solving, anti-detection browser tricks, scraping behind
```

```
login, or credential-driven automation unless a human architect explicitly marks
that path as approved and policy-compliant.

3. Build a secure multi-tenant-ready architecture by default.
   - Every persisted domain object must include tenant_id or equivalent
workspace isolation.
   - Apply tenant isolation at the database policy layer and the application
authorization layer.
   - Secrets, logs, queues, files, caches, and analytics must remain tenant-
scoped.

4. Build for production operations, not demo theater.
   - Include CI/CD, secret management, logging, metrics, tracing, alerting,
auditability, rollback, retry policies, idempotency keys, and approval gates.
   - Include staging and production environments.
   - Provide infrastructure-as-code and deployment instructions.

MISSION

Create a secure, deployable “AI automation revenue engine” that covers:
- frontend
- backend
- integrations
- analytics
- CRM
- lead capture
- campaign automation
- revenue tracking
- billing
- reporting
- optional multi-tenant support
- operator/admin controls

REQUIRED INPUTS

Use this exact input schema and ask for missing values in structured form only:

PROJECT_INPUTS = {
  organization_name: string,
  project_name: string,
  environment_names: [ "dev", "staging", "prod" ],
  website_urls: string[],
  domains_and_dns: {
    registrar: string,
    dns_provider: string,
    domains: string[]
  },
  repository_access: {
```

```
  repo_url: string,
  default_branch: string
},
tenant_model: {
  mode: "single-tenant" | "multi-tenant" | "single-now-multi-later",
  tenant_list: [
    {
      tenant_slug: string,
      tenant_name: string,
      region: string,
      currency: string,
      brand_voice_summary: string
    }
  ]
},
secret_references: {
  website_admin_secret_ref: string | null,
  social_oauth_secret_refs: string[],
  openai_api_key_ref: string | null,
  anthropic_api_key_ref: string | null,
  google_gemini_api_key_ref: string | null,
  xai_api_key_ref: string | null,
  nvidia_api_key_ref: string | null,
  glm_api_key_ref: string | null,
  crm_secret_refs: string[],
  billing_secret_refs: string[],
  analytics_secret_refs: string[],
  cloud_secret_refs: string[]
},
integrations: {
  crm: [ "salesforce" | "hubspot" | "pipedrive" | "zoho" | "custom" ],
  billing: [ "stripe" | "custom" ],
  email: [ "resend" | "sendgrid" | "ses" | "postmark" | "custom" ],
  sms: [ "twilio" | "custom" ],
  socials: [ "linkedin" | "x" | "meta" | "tiktok" | "youtube" | "custom" ],
  analytics: [ "ga4" | "server-side-events" | "warehouse-only" | "custom" ]
},
compliance_and_governance: {
  data_residency_requirements: string,
  retention_policy: string,
  approval_required_actions: string[],
  prohibited_actions: string[],
  pii_policy_summary: string
},
business_goals: {
  primary_kpis: string[],
  target_response_sla_minutes: number,
  target_mql_to_sql_rate: number | null,
```

```
    target_roas: number | null,
    target_pipeline_created: number | null
  }
}

DELIVERABLES

Produce all of the following:

A. EXECUTIVE SUMMARY
- Architecture decision summary
- Deployment summary
- Security summary
- Model routing summary
- Revenue operations summary

B. IMPLEMENTATION PLAN
- repo structure
- service boundaries
- database schema outline
- tenant isolation approach
- auth/RBAC approach
- secrets/vault approach
- queue/workflow approach
- observability approach
- CI/CD approach
- backup/restore approach
- disaster recovery approach

C. TECH STACK DECISION
Use the following default stack unless there is a compelling reason to change:
- Frontend: Next.js + React + TypeScript + Tailwind + component library
- Backend/API: Node.js + TypeScript
- Durable workflows: Temporal
- Transaction DB: PostgreSQL
- Cache/queue: Redis
- Analytics/event store: ClickHouse
- Files: S3-compatible object storage
- Billing: Stripe
- Auth: OAuth/SSO + WebAuthn/passkeys where possible
- Observability: OpenTelemetry + Sentry + Prometheus
- Secrets: HashiCorp Vault or cloud-native secrets manager
- CI/CD: GitHub Actions with OIDC
- IaC: Terraform

D. AI SYSTEM DESIGN
Implement a multi-model router with:
- task-based routing
```

```
- cost ceilings
- latency ceilings
- fallback providers
- output schema validation
- prompt-injection defenses
- PII redaction before logging
- human approval for high-risk outputs
- eval hooks for continuous improvement

Task classes:
- extraction / classification
- lead scoring
- content drafting
- email drafting
- campaign planning
- dashboard summarization
- internal coding assistance
- retrieval / grounding

Embeddings:
- default to retrieval before fine-tuning
- support pgvector or external vector indexing
- allow pluggable embedding provider

Fine-tuning:
- do not make fine-tuning a dependency for MVP
- document when fine-tuning would be justified

E. FRONTEND
Generate:
- public lead capture pages
- operator admin console
- tenant switcher
- approval queue
- AI router settings page
- secrets/integrations status page
- KPI dashboard
- audit log viewer
- revenue attribution dashboard

F. BACKEND
Generate:
- REST or route-handler endpoints
- webhook receivers
- workflow triggers
- idempotent job handlers
- provider adapters
- CRM adapters
```

```
- analytics event pipeline
- billing event pipeline
- tenant-aware repositories/services
- audit event writer

G. SECURITY CONTROLS
Implement and document:
- RBAC
- MFA/WebAuthn support
- tenant isolation
- least privilege
- vault-only secret access
- secret rotation playbook
- immutable audit logs
- webhook signature verification
- content security policy
- rate limits
- WAF-ready deployment assumptions
- prompt-injection defenses
- PII scrubbing
- safe logging policy
- incident response hooks

H. DEPLOYMENT
Provide:
- local dev setup
- staging deployment
- production deployment
- migration plan
- environment variable contract
- GitHub Actions workflow
- Terraform skeleton
- domain and DNS checklist
- rollback strategy
- smoke test plan

I. NVIDIA PATH
Include:
- hosted NVIDIA API prototyping path
- self-hosted NVIDIA NIM path
- when to use each
- GPU assumptions
- admin settings for enabling/disabling NVIDIA routes

J. OUTPUT FORMAT
Return:
1. architecture overview
2. mermaid diagrams
```

```
3. repo tree
4. database schema summary
5. API contract summary
6. frontend wireframes
7. prioritized roadmap
8. implementation checklist
9. effort/cost ranges
10. operator runbook
11. code samples in TypeScript/JavaScript
12. unresolved risks and assumptions

QUALITY BAR

- Prefer secure defaults over convenience.
- Prefer official APIs over credential tricks.
- Prefer durable workflows over cron spaghetti.
- Prefer tenant-scoped policy enforcement over app-only filtering.
- Prefer structured JSON outputs for machine actions.
- Prefer explicit costs, rate limits, and approvals.
- Prefer implementation artifacts that a real engineering team can ship.

Now generate the full solution.
```

Deployment, security, and operations

A production deployment should move in a straight line:

- **Provision identity and secret handling first.** Stand up Vault or Secrets Manager, define environment scopes, and create service accounts. Use GitHub Actions OIDC so deploy jobs receive short-lived cloud access rather than stored cloud keys. [25]
- **Provision data plane second.** Create PostgreSQL, Redis, object storage, and ClickHouse. Enable RLS before exposing any tenant-facing API surface. [26]
- **Deploy the web and API plane third.** Put the Next.js UI on Vercel or your chosen frontend host; place the API/workers on ECS, Cloud Run, or Azure Container Apps depending on your enterprise default. [27]
- **Deploy durable workers fourth.** Temporal-backed workflow workers should own retries, schedules, approvals, webhook fan-out, and recovery logic, not cron jobs hidden in app servers. [21]
- **Connect billing and async integrations fifth.** Stripe webhooks should verify signatures, return quick `2xx` responses, and push events into the workflow layer and analytics plane. [19]
- **Enable AI providers last.** Start with one fast path and one quality path, then add xAI or NVIDIA lanes. NVIDIA's current docs support both free hosted prototyping paths and self-hosted NIM deployments. [28]

For security and compliance, the defaults should be explicit rather than aspirational. OWASP's secrets, MFA, logging, and prompt-injection guidance maps cleanly to this system: centralize secrets; use MFA and passkeys for operators; avoid logging tokens, prompts with secrets, or raw PII; keep system prompts and retrieved content separated; validate outputs before executing tool actions; and require human approval for

external publishing, billing changes, destructive operations, or any workflow that could materially affect customers or revenue. [29]

The observability stack should be layered. Next.js has an OpenTelemetry guide; OpenTelemetry's Node.js docs cover instrumentation patterns; Sentry provides Next.js-specific application monitoring; Prometheus remains the standard metrics plane; and ClickHouse is well suited for large-scale events, logs, and traces. That combination lets you answer both "did the system fail?" and "did the automations create revenue?" [30]

The KPI set should be split into four groups so operators do not drown in vanity metrics:

- **Acquisition:** lead volume, qualified lead rate, source quality, cost per lead.
- **Pipeline:** MQL→SQL rate, meeting-booked rate, opportunity creation, pipeline created, pipeline velocity.
- **Revenue:** CAC, payback period, assisted revenue, won revenue, expansion revenue, failed-payment recovery.
- **Automation health:** lead response SLA, approval queue age, workflow success rate, fallback rate, token cost per successful workflow, webhook validation failures, secrets nearing rotation, RLS policy violations detected.

Deliverables for engineering leadership

Implementation checklist

- Freeze the input contract and ban plaintext secret handling.
- Choose hosting baseline: Vercel + AWS ECS/Fargate is the strongest general default.
- Stand up Vault or cloud-native secrets management and GitHub OIDC.
- Create PostgreSQL schemas with `tenant_id` and RLS policies.
- Add Redis, object storage, and ClickHouse.
- Stand up Temporal and define workflow primitives: `captureLead`, `qualifyLead`, `draftCampaign`, `approveAction`, `publishAction`, `recordRevenue`.
- Implement Auth.js/SSO and operator RBAC.
- Build the Next.js admin shell and KPI dashboard.
- Connect Stripe webhooks and revenue attribution events.
- Implement AI routing, schema validation, and approval gates.
- Add observability, audit logging, synthetic tests, and incident alerts.
- Rotate all onboarding credentials after go-live.

Prioritized roadmap

Horizon	What to ship	Why it comes here
Immediate	Secure input contract, vaulting, tenant model, lead capture, CRM sync, Stripe webhooks, base dashboards	These are the foundations of trust and measurable revenue impact

Horizon	What to ship	Why it comes here
Near-term	AI router, campaign drafting, approval queue, automated follow-up, attribution model, operator console	This unlocks visible automation without losing control
Mid-term	Social publishing via approved APIs, advanced scoring, retrieval layer, self-serve tenant onboarding, multi-region support	Expands scale and reduces per-project operational overhead
Later	NVIDIA self-hosted inference lanes, advanced experimentation/evals, model distillation, agent-on-agent workflows, selective fine-tuning	These are optimization layers, not prerequisites for revenue automation

Estimated effort and cost ranges

These are planning estimates, not vendor guarantees.

Scope	Team shape	Calendar estimate	Build cost range	Monthly run cost range
Foundation MVP	1 FE, 1 BE, 1 platform/full-stack, fractional product/design	6–10 weeks	$80k–$180k	$2k–$8k
Production v1	2–3 app engineers, 1 platform engineer, 1 data/analytics engineer, fractional design/QA	12–20 weeks	$220k–$650k	$8k–$35k
Enterprise hardening	Add security, data, and SRE capacity	+6–12 weeks	+$120k–$300k	+$5k–$25k

A useful AI-cost sanity check for **100 projects** is to model one month at **5M input and 1M output tokens per project**. At the current published rates, that lands roughly around `$2,750` for GPT-5.4, `$3,000` for Claude Sonnet-class pricing at $3/$15, `$1,650` for Gemini 3.5 Flash, and `$875` for Grok 4.3, before embeddings, search-grounding charges, retries, images/video, or caching savings. These are illustrative calculations based on currently published pricing pages, not negotiated enterprise contracts. [31]

Operator runbook

Purpose: keep the engine safe, measurable, and revenue-positive every day.

Start of day

- Check incident banner, webhook failures, workflow backlog, and approval queue age.
- Verify no provider route is breaching budget or latency thresholds.
- Review failed or stalled lead-response workflows first.

Before approving outbound actions

- Confirm correct tenant/workspace.
- Confirm audience, brand voice, and legal policy.
- Confirm destination channel and spend guardrails.
- Confirm the action has a rollback or correction path.

If a workflow fails

- Inspect trace, audit record, tenant, provider, and external dependency.
- Retry only idempotent steps.
- If failure involves billing, publishing, or deletion, require human review before rerun.

If a secret is exposed or suspected compromised

- Revoke the secret in the vault/provider.
- Rotate dependent credentials.
- Pause affected workflows.
- Review audit logs for blast radius.
- Re-enable only after replacement secret passes smoke tests.

Weekly

- Review automation KPIs vs pipeline and revenue KPIs.
- Inspect fallback rate, token cost per workflow, and low-performing prompts.
- Archive or delete expired artifacts per retention policy.
- Rotate temporary onboarding credentials and remove unused integrations.

Open questions and limitations

A few items should be validated by the engineering team before locking the build:

- **GLM/Z.ai** was included as an optional configuration slot, but the public official documentation path was not consistently available enough in this research session to justify making it a default production dependency.
- **Meta and TikTok official docs** were not reliably accessible through the browsing session, so this report deliberately recommends the broader secure pattern—OAuth and official APIs where available—without overcommitting to specific endpoints from those platforms.
- **CRM choice** materially changes object mapping, lead lifecycle, and attribution semantics. The architecture above supports Salesforce, HubSpot, or similar platforms, but the exact schema adapters should be generated from the chosen CRM's current API contracts.

The core conclusion does not change: the strongest deliverable for your developers is a **single master prompt plus a typed security-first implementation spec** that treats secrets as vaulted references, uses official integrations, isolates tenants at the data-policy layer, routes work across multiple models with explicit fallbacks, and measures revenue impact end to end. [32]

---

1 10 22 https://platform.openai.com/docs/guides/responses-vs-chat-completions
https://platform.openai.com/docs/guides/responses-vs-chat-completions

2 8 26 https://www.postgresql.org/docs/current/ddl-rowsecurity.html
https://www.postgresql.org/docs/current/ddl-rowsecurity.html

3 5 29 32 https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

4 24 https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html
https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html

6 https://authjs.dev/getting-started
https://authjs.dev/getting-started

7 https://learn.microsoft.com/en-us/linkedin/marketing/
https://learn.microsoft.com/en-us/linkedin/marketing/

9 https://nextjs.org/docs/app/guides/multi-tenant
https://nextjs.org/docs/app/guides/multi-tenant

11 https://platform.claude.com/docs/en/about-claude/models/overview
https://platform.claude.com/docs/en/about-claude/models/overview

12 https://ai.google.dev/gemini-api/docs/models
https://ai.google.dev/gemini-api/docs/models

13 https://docs.x.ai/docs/models
https://docs.x.ai/docs/models

14 https://docs.api.nvidia.com/
https://docs.api.nvidia.com/

15 27 https://vercel.com/docs/deployments
https://vercel.com/docs/deployments

16 https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html
https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html

17 https://cloud.google.com/run/docs/overview/what-is-cloud-run
https://cloud.google.com/run/docs/overview/what-is-cloud-run

18 https://learn.microsoft.com/en-us/azure/container-apps/overview
https://learn.microsoft.com/en-us/azure/container-apps/overview

19 https://docs.stripe.com/webhooks
https://docs.stripe.com/webhooks

20 https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html
https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html

21 https://docs.temporal.io/
https://docs.temporal.io/

23 https://developers.openai.com/api/docs/guides/fine-tuning
https://developers.openai.com/api/docs/guides/fine-tuning

25 https://docs.github.com/en/actions/deployment/security-hardening-your-deployments/about-security-hardening-with-openid-connect
https://docs.github.com/en/actions/deployment/security-hardening-your-deployments/about-security-hardening-with-openid-connect

28 https://developer.nvidia.com/nim
https://developer.nvidia.com/nim

30 https://nextjs.org/docs/app/guides/open-telemetry
https://nextjs.org/docs/app/guides/open-telemetry

31 https://openai.com/api/pricing/
https://openai.com/api/pricing/